Assignment for Test-Train Splitting & Polynomial Regression

Dataset: alloy-confp-train-data.csv

This PDF shows the notebook code directly so it is easy to read and submit.

Assignment for Test-Train Splitting & Polynomial Regression Using the provided dataset for the hardness of high-entropy alloys and their metal concentrations.

Test-Train Splitting

Code cell

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures

df = pd.read_csv('alloy-confp-train-data.csv')
print('Shape:', df.shape)
df.head()
```

Code cell

```
X_cols = ['Al', 'Co', 'Cr', 'Cu', 'Fe', 'Ni']
y_cols = ['HV']

X = df[X_cols].to_numpy()
y = df[y_cols].to_numpy().ravel()

print('X shape:', X.shape)
print('y shape:', y.shape)
```

Code cell

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=True, random_state=42
)

print('X_train shape:', X_train.shape)
print('X_test shape:', X_test.shape)
print('y_train shape:', y_train.shape)
```

```
print('y_test shape:', y_test.shape)
```

Code cell

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

y_pred = lr_model.predict(X_test)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print('Linear Regression RMSE:', rmse)
print('Linear Regression R2:', r2)
```

Code cell

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.8, label='Predicted values')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label='Ideal fit line')
plt.xlabel('Actual HV')
plt.ylabel('Predicted HV')
plt.title('Actual vs Predicted HV - Linear Regression')
plt.legend()
plt.show()
```

Polynomial Regression Now the same procedure is repeated after adding polynomial features up to degree 3.

Code cell

```
poly = PolynomialFeatures(degree=3, include_bias=False)
X_poly = poly.fit_transform(X)

print('Original X shape:', X.shape)
print('Polynomial X shape:', X_poly.shape)
```

Code cell

```
X_poly_train, X_poly_test, y_poly_train, y_poly_test = train_test_split(
    X_poly, y, test_size=0.2, shuffle=True, random_state=42
)

print('X_poly_train shape:', X_poly_train.shape)
print('X_poly_test shape:', X_poly_test.shape)
```

Code cell

```
poly_model = LinearRegression()
poly_model.fit(X_poly_train, y_poly_train)

y_poly_pred = poly_model.predict(X_poly_test)
poly_rmse = root_mean_squared_error(y_poly_test, y_poly_pred)
poly_r2 = r2_score(y_poly_test, y_poly_pred)

print('Polynomial Regression RMSE:', poly_rmse)
```

```
print('Polynomial Regression R2:', poly_r2)
```

Code cell

```
plt.figure(figsize=(8, 6))
plt.scatter(y_poly_test, y_poly_pred, color='green', alpha=0.8, label='Predicted values')
plt.plot([y_poly_test.min(), y_poly_test.max()], [y_poly_test.min(), y_poly_test.max()], 'r--', label='Ideal
plt.xlabel('Actual HV')
plt.ylabel('Predicted HV')
plt.title('Actual vs Predicted HV - Polynomial Regression (Degree 3)')
plt.legend()
plt.show()
```

Conclusion For this dataset, the test results show that linear regression performs better than degree-3 polynomial regression. The linear model gives RMSE = 71.96 and R2 = 0.83, while the polynomial model gives RMSE = 295.11 and R2 = -1.85. So the simpler linear model is the better fit here.